WEEK – 01

Implement the following Attacks:

a) DictionaryAttack

```
def dictionary_attack(target_password, file_path="wordlist.txt"):
    try:
        with open(file_path, "r", encoding="utf-8", errors="ignore") as file:
            for word in file:
                clean_word = word.strip()
                if clean_word == target_password:
                    return clean_word
    except FileNotFoundError:
        print(f"Error: Could not find '{file_path}'")
        return None
    return None

target = input("Enter target password: ").strip()
result = dictionary_attack(target, "wordlist.txt")

if result:
    print(f"Password found: {result}")
else:
    print("Password not found in wordlist.")
```

```
PS E:\Labs\sem 7\Cyber Security> python dictionary_attack.py
Enter target password: password123
Password found: password123
PS E:\Labs\sem 7\Cyber Security> python dictionary_attack.py
Enter target password: akLil982@(8
Password not found in wordlist.
```

b) Brute Force Attack

```
import itertools
import string
def brute_force_attack(target_password, max_length=4,
max_attempts=50):
    charset = string.ascii_lowercase + string.digits
    attempts = 0
    for length in range(1, max_length + 1):
        for candidate in itertools.product(charset, repeat=length):
            attempts += 1
            guess = ''.join(candidate)
            if guess == target_password:
                print(f"Match found: {guess}")
                print(f"Attempts: {attempts}")
                return guess
            if attempts >= max_attempts:
                print(f"Stopped: Reached maximum attempt limit of
{max_attempts}.")
                return None
```

```
target = input("Enter target password: ").strip()
brute_force_attack(target_password=target, max_length=4,
max_attempts=50)
```

```
PS E:\Labs\sem 7\Cyber Security> python .\brute_force_attack.py
Enter target password: aaaa
Match found: aaaa
Attempts: 47989
PS E:\Labs\sem 7\Cyber Security> python .\brute_force_attack.py
Enter target password: bHliLiik
```

WEEK – 02

Write a program for static malware analysis that performs file signature checking, string extraction, and entropy analysis to detect suspicious files.

```
import math
import re
from collections import Counter
def file_signature_check(file_path):
    signatures = {
        b"MZ": "Windows Executable (PE)",
        b"\x7FELF": "Linux Executable (ELF)",
        b"PK": "ZIP Archive",
        b"%PDF": "PDF File"
    }

    with open(file_path, "rb") as f:
        header = f.read(4)
    print("File Signature Check:")
    found = False
    for sig, name in signatures.items():
        if header.startswith(sig):
            print("Signature Matched:", name)
            found = True
            break
    if not found:
        print("Unknown File Signature")
def extract_strings(file_path, min_length=4):
    with open(file_path, "rb") as f:
        data = f.read()
    strings = re.findall(rb"[\x20-\x7E]{%d,}" % min_length, data)
    print("\nExtracted Strings:")
    for s in strings:
        print(s.decode(errors="ignore"))

def calculate_entropy(file_path):
    with open(file_path, "rb") as f:
        data = f.read()
    if len(data) == 0:
        return 0
    counter = Counter(data)
    entropy = 0
    for count in counter.values():
        p = count / len(data)
        entropy -= p * math.log2(p)
    print("\nEntropy Analysis:")
    print("Entropy:", round(entropy, 4))

    if entropy > 7.0:
        print("High entropy detected. File may be packed or encrypted.")
    else:
        print("Normal entropy.")
```

```
file_path = input("Enter file path: ")

file_signature_check(file_path)
extract_strings(file_path)
calculate_entropy(file_path)
```

```
PS E:\Labs\sem 7\Cyber Security> python .\static_malware_analysis.py
Enter file path: 22CS11S5.pdf
=== File Signature Check ===
Signature Matched: PDF File

=== Extracted Strings (Top 10 Displayed) ===
Total Strings Extracted: 2492
 -> %PDF-1.5
 -> 29 0 obj
 -> <</Linearized 1 /L 204515 /H [ 1201 157 ] /O 31 /E 7440 /N 2 /T 203740 >>
 -> endobj
 ->
 -> xref
 -> 29 19
 -> 0000000017 00000 n
 -> 0000001358 00000 n
 -> 0000001438 00000 n

=== Entropy Analysis ===
Entropy: 7.9464 / 8.0
High entropy detected. File may be packed or encrypted.
```

WEEK – 03

Write a program that reads an email body (as a string) and flags it as spam if it contains typical spam keywords like “lottery”, “free”, “click here”, etc.

```
def check_spam(email):
    spam_keywords = [
        "lottery",
        "free",
        "click here",
        "winner",
        "claim",
        "prize",
        "money",
        "offer",
        "urgent",
        "congratulations"
    ]
    email = email.lower()
    found = []
    for word in spam_keywords:
        if word in email:
            found.append(word)
    if found:
        print("Spam Email Detected!")
        print("Spam Keywords Found:", found)
    else:
        print("Email appears to be Safe.")
email = input("Enter Email Body:\n")
check_spam(email)
```

```
PS E:\Labs\sem 7\Cyber Security> python spam_detection.py
Enter Email Body:
If you do not verify your details within 24 hours, your account will be
Email appears to be Safe.
PS E:\Labs\sem 7\Cyber Security> python spam_detection.py
Enter Email Body:
You have officially won $5,000,000.00 USD in our International Free Mega
users.
Spam Email Detected!
Spam Keywords Found: ['lottery', 'free']
```

WEEK - 04

Write a program to

a) Check the strength of a password based on uppercase, lowercase, digits, special characters, and length.

b) Hash the password using bcrypt and save it to a file.

c) Create a program to verify a login password by comparing it with the stored hash.

```
password = input("Enter Password: ")
uppercase = 0
lowercase = 0
digit = 0
special = 0
for ch in password:
    if 'A' <= ch <= 'Z':
        uppercase = 1
    elif 'a' <= ch <= 'z':
        lowercase = 1
    elif '0' <= ch <= '9':
        digit = 1
    else:
        special = 1
length = len(password)
print("\nPassword Analysis")
print("Length:", length)
if length >= 8 and uppercase and lowercase and digit and special:
    print("Password Strength: Strong")
elif length >= 6 and (uppercase + lowercase + digit + special) >= 3:
    print("Password Strength: Medium")
else:
    print("Password Strength: Weak")


import bcrypt
password = input("Enter Password: ")
password = password.encode()
hashed_password = bcrypt.hashpw(password, bcrypt.gensalt())
with open("password.txt", "wb") as file:
    file.write(hashed_password)
```

```
print("\nPassword hashed successfully!")
print("Hash stored in password.txt")

import bcrypt
password = input("Enter Login Password: ")
password = password.encode()

with open("password.txt", "rb") as file:
    stored_hash = file.read()

if bcrypt.checkpw(password, stored_hash):
    print("\nLogin Successful!")
else:
    print("\nInvalid Password!")
```

```
PS E:\Labs\sem 7\Cyber Security> python password_strength.py
Enter Password: hello123

Password Analysis
Length: 8
Password Strength: Weak
Enter Password: kJJ)8&&#1254

Password hashed successfully!
Hash stored in password.txt
Enter Login Password: kJJ)8&&#1254

Login Successful!
PS E:\Labs\sem 7\Cyber Security> python password_strength.py
Enter Password: hii

Password Analysis
Length: 3
Password Strength: Weak
Enter Password: abdefka

Password hashed successfully!
Hash stored in password.txt
Enter Login Password: ;lkj

Invalid Password!
```

WEEK – 05

Write a script to interact with a security-related API (e.g., VirusTotal) to retrieve threat intelligence or breach data.

```
import requests
API_KEY =
"0e070ada999941d2583269ad4afa9ef877e80248ed0c33492c3a42f824d75b28"
file_hash =
"59e2faae4cc09fb0d10009b096506d4d0dc0b459abbc7c3bc0c3fba65cc90255"
url = f"https://www.virustotal.com/api/v3/files/{file_hash}"
headers = {
    "x-apikey": API_KEY
}
response = requests.get(url, headers=headers)
if response.status_code == 200:
    data = response.json()
    attributes = data["data"]["attributes"]
    stats = attributes["last_analysis_stats"]
    print("========== VirusTotal Report ==========")
    print("File Name      :", attributes["meaningful_name"])
    print("File Type      :", attributes["type_description"])
    print("File Size      :", attributes["size"], "bytes")
    print("SHA-256        :", attributes["sha256"])
    print("MD5            :", attributes["md5"])
    print("SHA-1          :", attributes["sha1"])
    print("\n----- Analysis Results -----")
    print("Malicious      :", stats["malicious"])
    print("Suspicious     :", stats["suspicious"])
    print("Harmless       :", stats["harmless"])
    print("Undetected     :", stats["undetected"])
else:
    print("Error:", response.status_code)
```

```
PS E:\Labs\sem 7\Cyber Security> python api_analysis.py
========== VirusTotal Report ==========
File Name      : 5.txt
File Type      : Text
File Size      : 26 bytes
SHA-256        : 59e2faae4cc09fb0d10009b096506d4d0dc0b459abbc7c3bc0c3fba65cc90255
MD5            : 30b40b34aba6e670571a8bf1095cb031
SHA-1          : b2be840a6435dcd1f5a24b19089a20ca86746dc5

----- Analysis Results -----
Malicious      : 0
Suspicious     : 0
Harmless       : 0
Undetected     : 61
```

WEEK – 06

Write a program to extract hyperlinks from a given website using web scraping. Modify the program to detect suspicious or phishing-related URLs for cybersecurity analysis.

```
import requests
from bs4 import BeautifulSoup
from urllib.parse import urljoin, urlparse
import re
# Function to analyze a URL
def analyze_url(url):
    print("\nURL:", url)
    suspicious = False

    # HTTP / HTTPS
    if url.startswith("https://"):
        print("Protocol : HTTPS (Secure)")
    elif url.startswith("http://"):
        print("Protocol : HTTP (Not Secure)")
        suspicious = True
    parsed = urlparse(url)
    domain = parsed.netloc

    # Check IP Address
    ip_pattern = r"^\d{1,3}(\.\d{1,3}){3}$"
    if re.fullmatch(ip_pattern, domain):
        print("IP Address : Found")
        suspicious = True
    else:
        print("Domain :", domain)

    # Check @
    if "@" in url:
        print("Contains '@' : Yes")
        suspicious = True

    # Suspicious Keywords
    keywords = [
        "login",
        "verify",
        "secure",
        "update",
        "bank",
        "account",
        "password"
    ]
```

```
found = False
    for word in keywords:
        if word in url.lower():
            print(f"Contains '{word}'")
            suspicious = True
            found = True

    if not found:
        print("No suspicious keywords")

    # Multiple underscores
    if url.count("_") > 1:
        print("Multiple underscores found")
        suspicious = True

    # Hyphen in domain
    if "-" in domain:
        print("Hyphen in domain")
        suspicious = True

    # Long URL
    if len(url) > 100:
        print("Very long URL")
        suspicious = True

    # Final Result
    if suspicious:
        print("Result : Suspicious URL")
    else:
        print("Result : Safe URL")

    print("-" * 60)
website = input("Enter Website URL: ")
try:
    response = requests.get(website, timeout=5)
    response.raise_for_status()
    soup = BeautifulSoup(response.text, "html.parser")
    print("\nExtracted Hyperlinks\n")
    links = soup.find_all("a", href=True)
    if not links:
        print("No hyperlinks found.")
    for tag in links:
        link = urljoin(website, tag["href"])
        analyze_url(link)
except requests.exceptions.RequestException:
    print("\nWebsite could not be reached.")
    print("Analyzing the entered URL instead...")
    analyze_url(website)
```

WEEK – 07

Demonstrate penetration testing tasks like port scanning, vulnerability scanning with Nmap tool.

1. Objective & Overview

The objective of this lab experiment is to demonstrate essential penetration testing tasks including active host discovery, targeted port scanning, and remote service/operating system version detection using Nmap. The experiments validate how network reconnaissance maps live hosts, identifies reachable transport layer ports, and fingerprints running service versions across local subnets, loopback adapters, and public test environments.

2. Experimental Execution & Screenshot Documentation

Task A: Network Host Discovery (Ping Sweep)

```
$ sudo nmap -sn 10.30.0.0/24
```

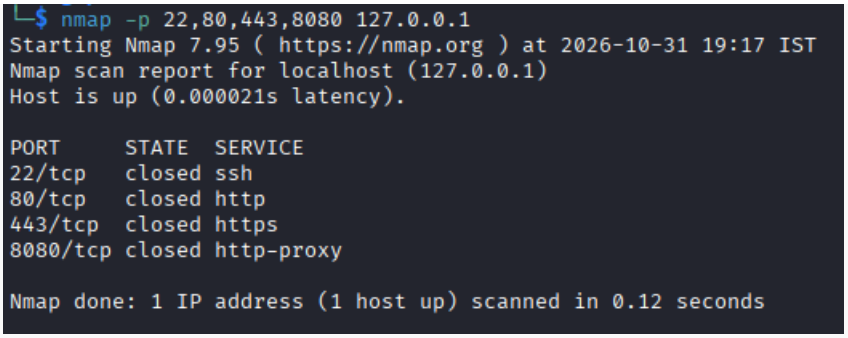
Description & Analysis:
• Command executed: sudo nmap -sn 10.30.0.0/24
• Purpose: Performs a Ping Sweep on the local 10.30.0.0/24 subnet to discover active hosts without conducting port scans.
• Privileges: Running with sudo grants raw packet privileges (ARP/ICMP discovery), making the sweep exceptionally fast.
• Observed Results: Discovered live active hosts including 10.30.0.1 (Sophos router), and multiple Netgear endpoints (10.30.0.50, 10.30.0.52, 10.30.0.54, 10.30.0.55, 10.30.0.57, 10.30.0.58, 10.30.0.59) with low latencies (~2-4 ms).

```
└─$ sudo nmap -sn 10.30.0.0/24
Starting Nmap 7.95 ( https://nmap.org ) at 2026-10-31 19:08 IST
Nmap scan report for 10.30.0.1
Host is up (0.0029s latency).
MAC Address: 7C:5A:1C:C9:6F:23 (Sophos)
Nmap scan report for 10.30.0.50
Host is up (0.0032s latency).
MAC Address: 94:A6:7E:73:F6:6F (Netgear)
Nmap scan report for 10.30.0.52
Host is up (0.0031s latency).
MAC Address: 94:A6:7E:73:E7:EF (Netgear)
Nmap scan report for 10.30.0.54
Host is up (0.0044s latency).
MAC Address: 94:A6:7E:73:DD:CF (Netgear)
Nmap scan report for 10.30.0.55
Host is up (0.0022s latency).
MAC Address: 94:A6:7E:73:DF:8F (Netgear)
Nmap scan report for 10.30.0.57
Host is up (0.0042s latency).
MAC Address: 94:A6:7E:73:DC:4F (Netgear)
Nmap scan report for 10.30.0.58
Host is up (0.0041s latency).
MAC Address: 94:A6:7E:73:EF:AF (Netgear)
Nmap scan report for 10.30.0.59
Host is up (0.0041s latency).
```

Task B1: Targeted Port Scan on Local Host (Loopback)

```
$ nmap -p 22,80,443,8080 127.0.0.1
```

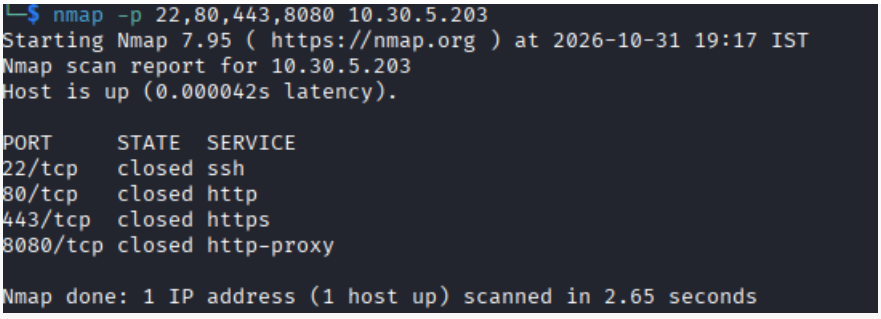
Description & Analysis:
• Command executed: nmap -p 22,80,443,8080 127.0.0.1
• Purpose: Evaluates specific standard service ports (SSH-22, HTTP-80, HTTPS-443, HTTP-Proxy-8080) on the local machine interface (127.0.0.1).
• Observed Results: All 4 targeted TCP ports were reported as 'closed'. This indicates the local host is active (0.000021s latency), but no web or SSH server daemons are actively listening on these ports.



Task B2: Targeted Port Scan on Remote Subnet Host

```
$ nmap -p 22,80,443,8080 10.30.5.203
```

Description & Analysis:
• Command executed: nmap -p 22,80,443,8080 10.30.5.203
• Purpose: Probes specific ports on a target remote IP address within the internal lab network.
• Observed Results: Host 10.30.5.203 confirmed online (2.65 seconds total scan time). All 4 queried ports (22, 80, 443, 8080) returned a 'closed' state, confirming no active services are bound to these ports.



Task C: Service Version & Operating System Fingerprinting

```
$ nmap -sV -O scanme.nmap.org
```

Description & Analysis:
• Command executed: nmap -sV -O scanme.nmap.org
• Target IP: 45.33.32.156 (IPv6: 2600:3c01::f03c:91ff:fe18:bb2f)
• Purpose: Detect exact software versions (-sV) and infer OS signature (-O) on open ports.
• Discovered Open Services:
- Port 22/tcp: Open | Service: SSH | Version: OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13 (Ubuntu Linux;

protocol 2.0)

- Port 25/tcp: Open | Service: SMTP
- Port 80/tcp: Open | Service: HTTP | Version: Apache httpd 2.4.7 ((Ubuntu))
- Port 9929/tcp: Open | Service: nping-echo | Version: Nping echo
- Filtered Ports: 2323/tcp (3d-nfsd), 50002/tcp (iiimsf)

```
└─$ nmap -sV -O scanme.nmap.org
Starting Nmap 7.95 ( https://nmap.org ) at 2026-10-31 19:20 IST
Nmap scan report for scanme.nmap.org (45.33.32.156)
Host is up (0.27s latency).
Other addresses for scanme.nmap.org (not scanned): 2600:3c01::f03c:91ff:fe18:bb2f
Not shown: 994 closed tcp ports (reset)
PORT      STATE    SERVICE    VERSION
22/tcp    open     ssh        OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13 (Ubuntu Linux; protocol 2.0)
25/tcp    open     smtp
80/tcp    open     http       Apache httpd 2.4.7 ((Ubuntu))
2323/tcp  filtered 3d-nfsd
9929/tcp  open     nping-echo Nping echo
50002/tcp filtered iiimsf
```

WEEK – 08

Installation of Network traffic monitoring tools (Wireshark/tcpdump/Ethereal) and observe data transferred in client server communication using UDP/TCP and identify the UDP/TCP datagram.

1. Aim & Objective

To set up client-server communication over TCP/UDP using Netcat (nc) and capture/analyze real-time network traffic using Wireshark. The experiment demonstrates interactive bidirectional communication between client and server nodes and inspects TCP flags, sequence numbers, and unencrypted payload data within captured datagrams.

2. Experimental Execution & Analysis

Step 1: Netcat Server (Listener Mode)

```
$ nc -lvnp 4444
```

• Command Breakdown: -l (listen), -v (verbose mode), -n (numeric IP without DNS resolution), -p 4444 (listen port).
• Observed Execution: The listener starts and awaits incoming connections. Once established from IP 10.30.5.203 via ephemeral port 51608, an interactive plaintext session opens. Messages received from the client ('hii', 'hello', 'cysec') are rendered live.

```
└─$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [10.30.5.203] from (UNKNOWN) [10.30.5.203] 51608
hii
hello
cysec
```

Step 2: Netcat Client (Initiator Connection)

```
$ nc 10.30.5.203 4444
```

• Command Breakdown: Connects directly to the server at IP 10.30.5.203 on TCP port 4444.
• Observed Execution: Upon connection, an unencrypted full-duplex socket is established. The operator inputs text stream strings ('hii', 'hello', 'cysec') which are immediately delivered across the TCP connection.

```
└─$ nc 10.30.5.203 4444
hii
hello
cysec
```

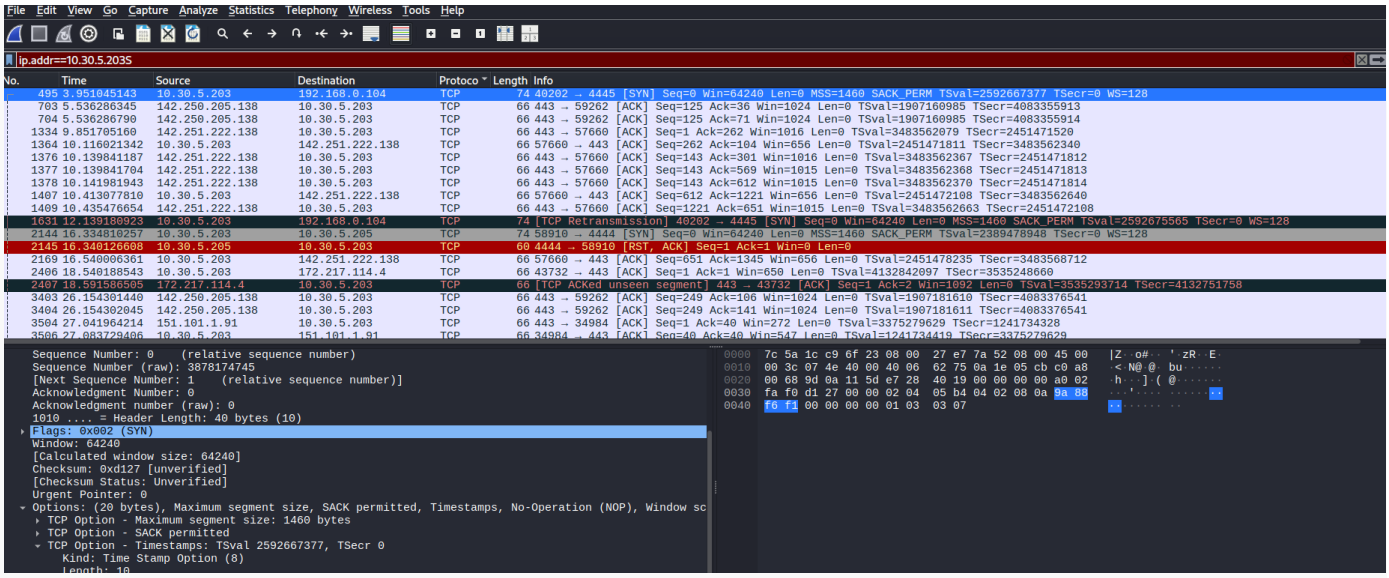
Step 3: Wireshark Network Traffic & Datagram Inspection

• Display Filter: ip.addr == 10.30.5.203 (or tcp.port == 4444)
• Packet Capture Observations:
1. TCP Connection / Reset Analysis: Packet 2145 shows a TCP RST, ACK segment sent from port 4444 to port 58910 (Win=0, Len=0), terminating an active socket session.

2. TCP Header Inspection: Detailed view of Packet 2144 (TCP SYN packet to port 4444 on 10.30.5.203). Window size set to 64240, MSS=1460 bytes, SACK permitted.

3. TCP Control Flags: Expanded Flags field shows 0x002 (SYN bit set), initiating the 3-Way Handshake (SYN -> SYN-ACK -> ACK).

4. Hex Packet Payload: The byte pane displays raw hexadecimal and ASCII plaintext bytes transferred across the transport layer.



WEEK - 09

Analyze Traffic Sniffing using ARP Poisoning.

Activity 1: Baseline and Poisoned ARP Neighbor Table (Ubuntu)

This screenshot shows the initial 'ip neigh' execution on the Ubuntu target VM before launching the

attack, followed by the second 'ip neigh' command demonstrating successful cache poisoning.

```
ubuntu@ubuntu:~$ ip neigh
10.30.0.1 dev enp0s3 lladdr 00:00:00:00:00:00 REACHABLE
10.30.5.130 dev enp0s3 lladdr 08:00:27:c3:fb:76 REACHABLE
ubuntu@ubuntu:~$ ip neigh
10.30.0.1 dev enp0s3 lladdr 08:00:27:c3:fb:76 REACHABLE
10.30.5.130 dev enp0s3 lladdr 08:00:27:c3:fb:76 REACHABLE
```

Figure 1: Ubuntu terminal output showing initial and poisoned ARP table states.

Analysis: Technical Analysis of Screenshot 1:

1. First execution of 'ip neigh' displays the legitimate default gateway (10.30.0.1) mapped to 00:00:00:00:00:00.

2. Second execution shows the gateway IP (10.30.0.1) updated to the Kali MAC address (08:00:27:c3:fb:76), confirming that ARP cache poisoning succeeded on the target VM.

Activity 2: ARP Spoof Execution and Cleanup (Kali Linux)

This screenshot demonstrates launching the arpspoof command from Kali Linux targeting the Ubuntu machine, followed by terminating the session with Ctrl+C to clean up and re-arp the targets.

```
└─$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group def
ault qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
roup default qlen 1000
    link/ether 08:00:27:c3:fb:76 brd ff:ff:ff:ff:ff:ff
    inet 10.30.5.130/21 brd 10.30.7.255 scope global dynamic noprefixroute et
h0
       valid_lft 10773sec preferred_lft 10773sec
    inet6 fe80::26a7:a3b3:cb11:650f/64 scope link noprefixroute
       valid_lft forever preferred_lft forever

┌──(kali㉿kali)-[~]
└─$ sudo arpspoof -i eth0 -t 10.30.4.213 10.30.0.1
8:0:27:c3:fb:76 8:0:27:d6:d9:ea 0806 42: arp reply 10.30.0.1 is-at 8:0:27:c3:
fb:76
8:0:27:c3:fb:76 8:0:27:d6:d9:ea 0806 42: arp reply 10.30.0.1 is-at 8:0:27:c3:
fb:76
8:0:27:c3:fb:76 8:0:27:d6:d9:ea 0806 42: arp reply 10.30.0.1 is-at 8:0:27:c3:
fb:76
^C
Cleaning up and re-arping targets...
8:0:27:c3:fb:76 8:0:27:d6:d9:ea 0806 42: arp reply 10.30.0.1 is-at 0:0:0:0:0:
0
```

Figure 2: Kali terminal output showing arpspoof running continuously and issuing ARP repair packets upon termination.

Analysis: Technical Analysis of Screenshot 2:

1. The command 'sudo arpspoof -i eth0 -t 10.30.4.213 10.30.0.1' continuously broadcasts unsolicited

ARP replies claiming 10.30.0.1 is at 08:00:27:c3:fb:76.

2. Pressing Ctrl+C stops the spoofing attack and triggers automated ARP restoration to prevent lingering

network issues.

WEEK – 10

Demonstrate DoS (Denial of Service) and DDoS (Distributed Denial of Service) attacks.

1. Experimental Environment & Target Setup

The target web server environment was prepared on the Ubuntu machine by initializing the Apache2 daemon and verifying network interfaces:

Host / Component Interface & IP Address Active Service / Tool

Target Web Server (Ubuntu) enp0s3 (10.30.4.213 /21) Apache2 HTTP Server (Port 80)

Traffic Generator (Kali Linux) eth0 / wlan0 (10.30.5.130 /21) ApacheBench (ab v2.3)

```
vboxuser@dLinux:~$ sudo systemctl start apache2
vboxuser@dLinux:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group defaul
t qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gr
oup default qlen 1000
    link/ether 08:00:27:d6:d9:ea brd ff:ff:ff:ff:ff:ff
    inet 10.30.4.213/21 brd 10.30.7.255 scope global dynamic noprefixroute enp0s
3
       valid_lft 7447sec preferred_lft 7447sec
    inet6 fe80::a00:27ff:fed6:d9ea/64 scope link
       valid_lft forever preferred_lft forever
vboxuser@dLinux:~$
```

Figure 1: Target host network configuration displaying active IP address 10.30.4.213 on interface enp0s3.

2. Stress Testing Execution & Traffic Analysis

Phase 1: Dynamic Query Stress Test (-n 5000 -c 500)

To test server capacity beyond simple static page retrieval, traffic was directed at an unindexed query path (/search?q=test) with 5,000 requests across 500 concurrent connections.

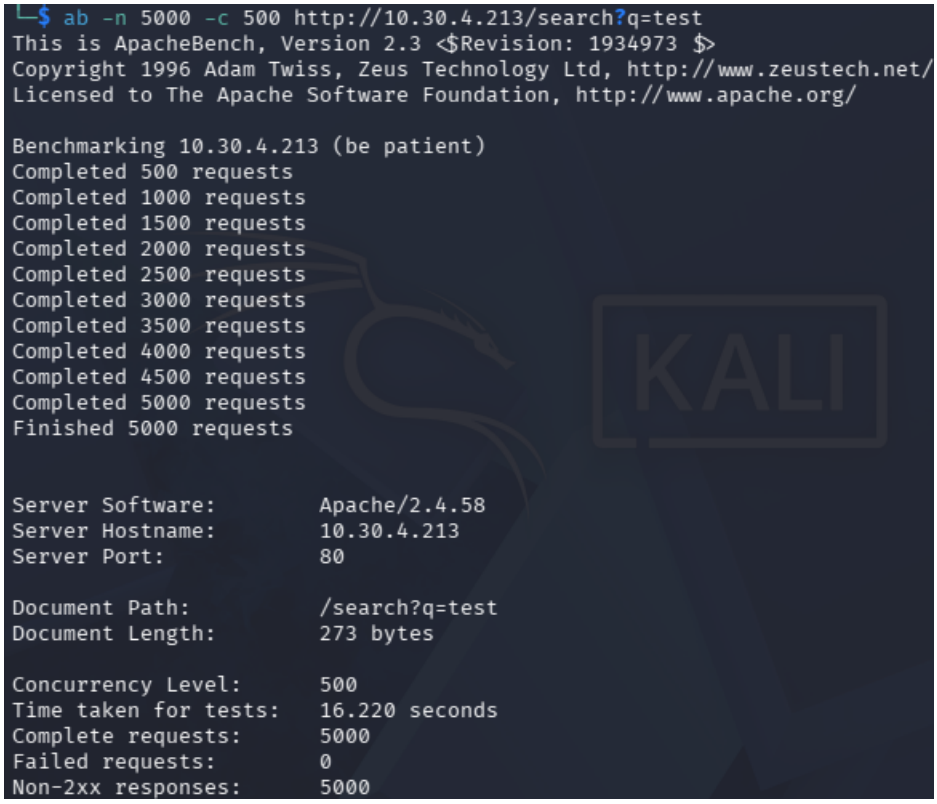


Figure 2: ApacheBench output for query path stress test showing non-2xx HTTP 404 responses handled cleanly over 16.22seconds.

Phase 1 Analysis: Observations & Analysis:

1. Complete Requests: All 5,000 requests were successfully completed without socket failures.

2. Non-2xx Responses: 5,000 responses returned HTTP 404 (Not Found), as the path /search?q=test was unindexed. The server correctly generated error responses for all incoming connections.

3. Failed Requests: 0 failed network requests, demonstrating that while error handling occurred, connection saturation was not achieved.

Phase 2: High-Concurrency Exhaustion Test (-n 10000 -c 1000)

To induce connection saturation, the concurrency was scaled to 1,000 simultaneous connections with a total volume of 10,000 requests directed at http://10.30.4.213/.

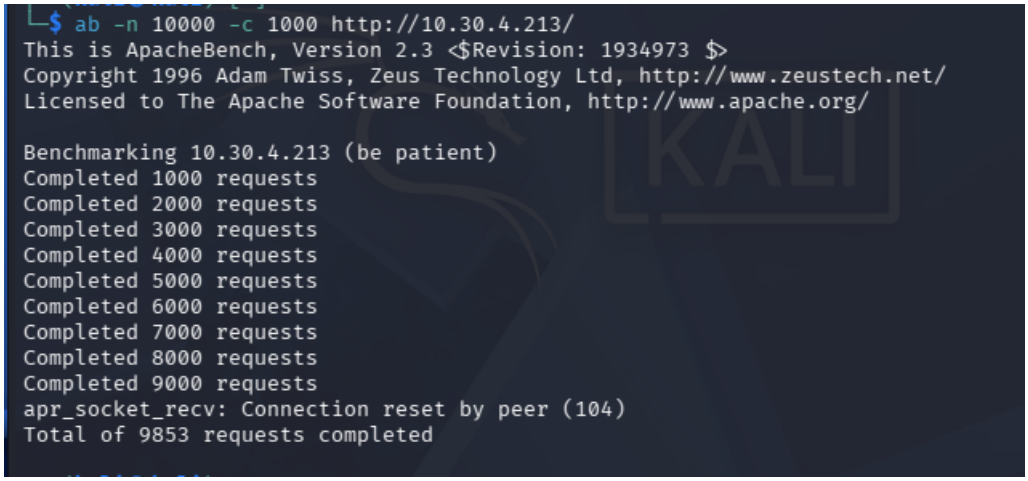


Figure 3: ApacheBench execution terminating prematurely with 'apr_socket_recv: Connection reset by peer (104)' after completing 9,853 requests.

Phase 2 Analysis: Observations & Analysis:

1. Unhandled Socket Failure: The benchmark failed with error 'apr_socket_recv: Connection reset by peer (104)'.

2. Service Exhaustion: Only 9,853 out of 10,000 requests were fulfilled before Apache2's worker process limit (MaxRequestWorkers) and kernel listen queue were completely overwhelmed, causing abrupt connection drops.

3. Lab Conclusion: This confirmed a successful Denial of Service (DoS) state, as legitimate request threads were dropped due to resource saturation.

WEEK – 11

Demonstrate SQL Injection, Cross-Site Scripting (XSS) and File Upload vulnerabilities in web applications.

The security assessment was executed using Damn Vulnerable Web Application (DVWA) deployed via Docker on Kali Linux, accessed locally at http://localhost:8080. Security settings were set to Low mode to analyze foundational vulnerability mechanisms.

Parameter Configuration Notes

Target Container vulnerables/web-dvwa Docker container listening on port 8080

Security Level Low Disables server-side sanitization for

vulnerability proof-of-concept



Figure 1: Terminal output confirming execution of 'sudo docker run --rm -it -p 8080:80 vulnerables/web-dvwa' and successful initialization of MariaDB and Apache services.

2. Vulnerability Demonstrations & Evidence

Activity 1: SQL Injection (Data Extraction)

The User ID query field was tested for SQL injection by submitting a boolean payload designed to evaluate true for all database records.

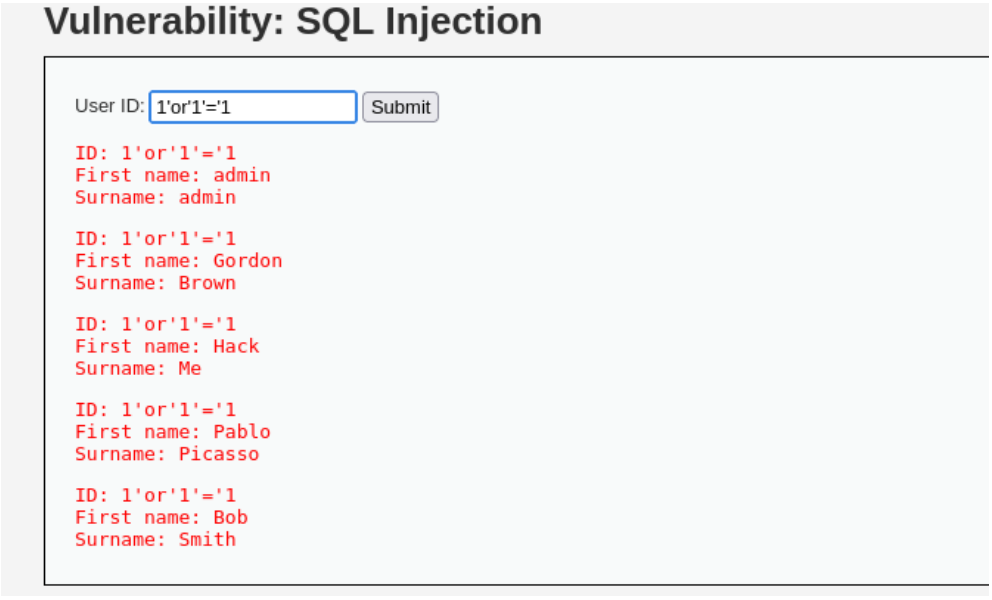


Figure 2: Submission of 1'or'1'='1 in DVWA returning all registered database records (admin, Gordon, Hack, Pablo, Bob).

Analysis: Technical Breakdown:

• Target Input Field: User ID

• Injected Payload: 1'or'1'='1

• Observed Result: The backend application concatenated the unvalidated string into the SQL query:

SELECT First_Name, Last_Name FROM users WHERE user_id = '1' or '1'='1'. Because '1'='1' evaluates to

true for every row in the database table, the application dumped the full account list.

Activity 2: Reflected Cross-Site Scripting (XSS)

Reflected XSS testing was conducted on both DVWA and OWASP Juice Shop to verify cross-application client-side execution.

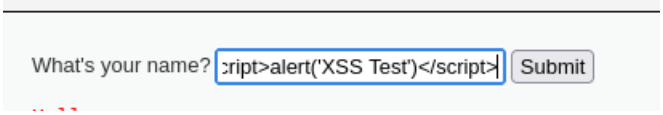


Figure 3: Injected script tag payload <script>alert('XSS Test')</script> submitted into DVWA name field.

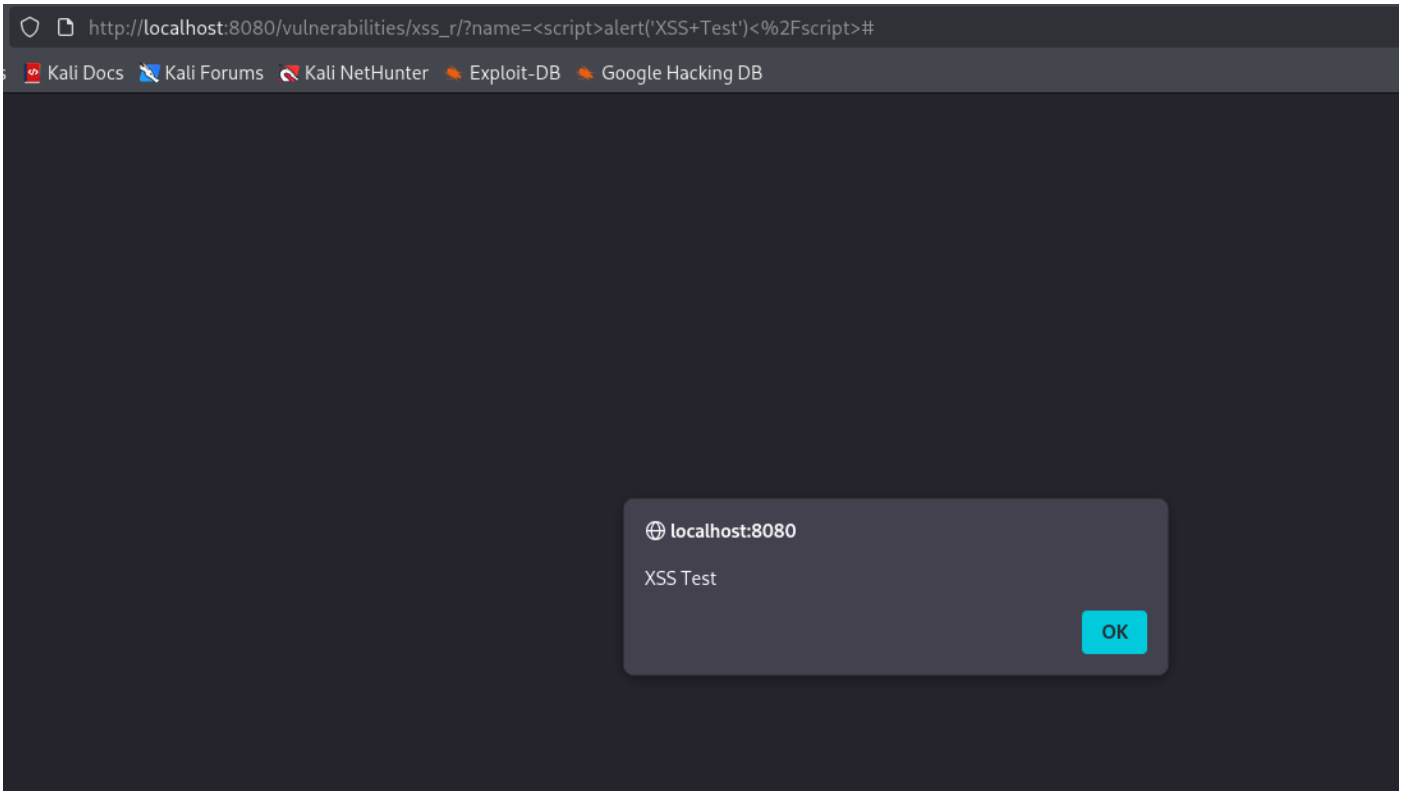


Figure 4: Search bar payload <iframe src="javascript:alert('SSD')"> executing a client-side alert box in Juice Shop.

Analysis: Technical Breakdown:

• DVWA Payload: <script>alert('XSS Test')</script>

• Juice Shop Payload: <iframe src="javascript:alert('SSD')">

• Observed Result: Both applications reflected user-controlled input directly into the HTML DOM without sanitization or HTML entity encoding, forcing the browser to execute client-side JavaScript pop-ups.

Activity 3: Unrestricted File Upload & Directory Traversal Execution

The upload functionality was assessed by submitting a non-image text file and verifying its storage and web accessibility.

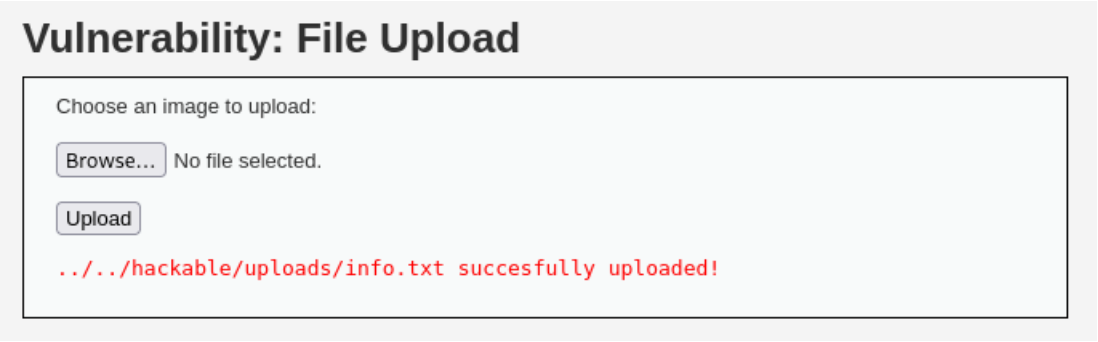


Figure 5: Confirmation screen showing text file info.txt uploaded to ../../hackable/uploads/info.txt.

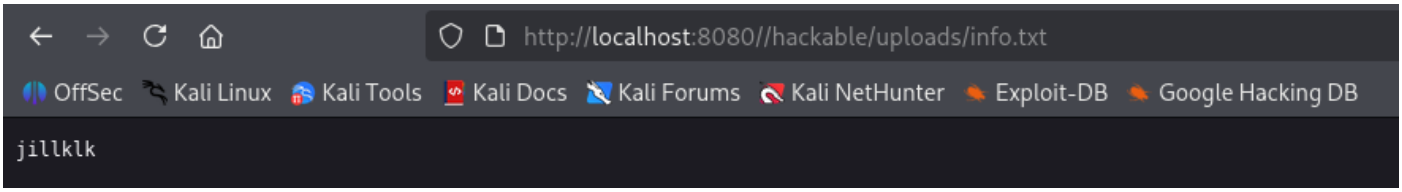


Figure 6: Navigating to http://localhost:8080//hackable/uploads/info.txt revealing file content ('jillklk').

Analysis: Technical Breakdown:

• File Uploaded: info.txt (Containing text data: 'jillklk')

• Relative Upload Path: ../../hackable/uploads/info.txt

• Access URL: http://localhost:8080//hackable/uploads/info.txt

• Observed Result: The application accepted a plain text format (.txt) without file extension or MIME-

WEEK – 12

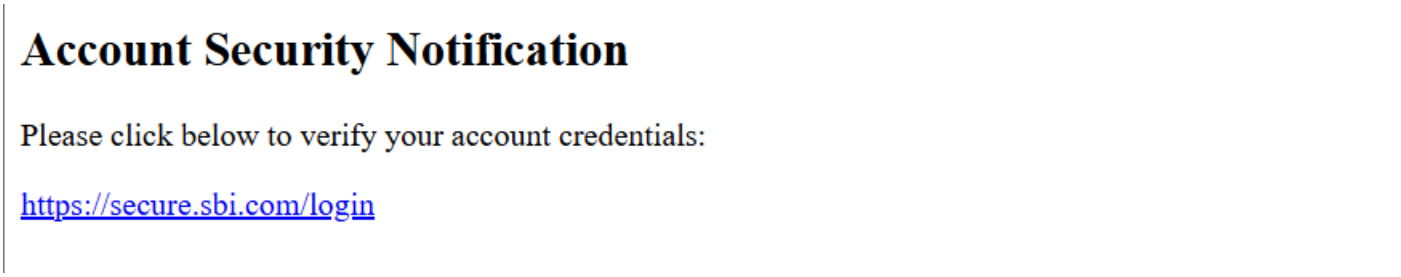
Illustrate the Phishing Attacks, Email Spoofing and Malware via Attachments vulnerabilities of E-Mail Application.

Activity 1: Phishing Link & Hyperlink Deception

Phishing emails frequently exploit the structural separation between display text rendered to the user and the actual destination URL embedded in the HTML anchor tag.

abc.html > html > body > a

```
<!DOCTYPE html>
<html>
<body>
  <h2>Account Security Notification</h2>
  <p>Please click below to verify your account credentials:</p>
  <a href="http://127.0.0.1:8080/fake-login">https://secure.sbi.com/login</a>
</body>
</html>
```



Account Security Notification

Please click below to verify your account credentials:

https://secure.sbi.com/login

Figure 1: Account Security Notification rendering a trusted display link (https://secure.sbi.com/login) to disguise underlying destination redirection.

Analysis: Technical Breakdown:

• Visual Display Text: https://secure.sbi.com/login

• Mechanism: HTML anchor tags (<a href="...">) allow arbitrary text to be displayed independently of

the actual href destination.

• Observed Result: Users reviewing only the visual text are deceived into trusting the message, while network traffic is directed to an attacker-controlled endpoint.

Activity 2: Malware Attachment Masking via Double Extensions

Attachers append secondary extension suffixes (e.g., .pdf.sh) to conceal executable scripts as standard static documents.

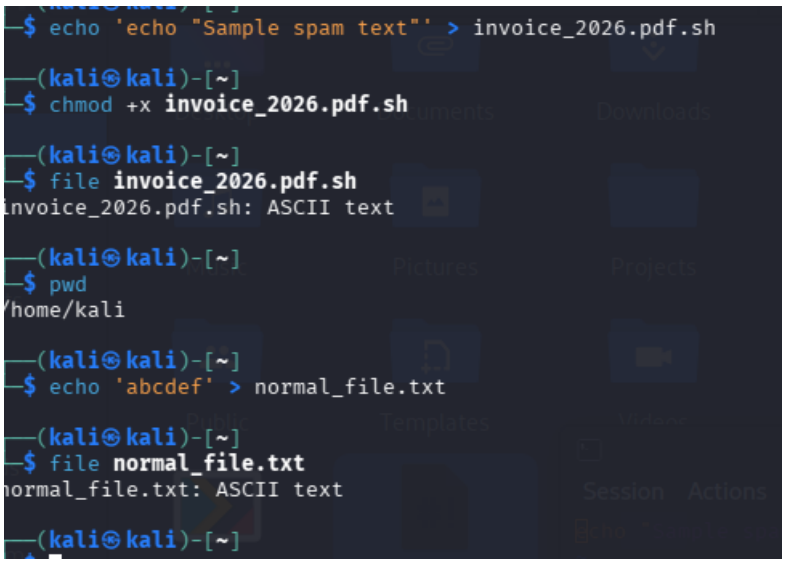


Figure 2: Kali Linux terminal running file invoice_2026.pdf.sh and file normal_file.txt, verifying both files evaluate as ASCII text.

Analysis: Technical Breakdown:

• Injected Filename: invoice_2026.pdf.sh

• Executed Command: file invoice_2026.pdf.sh

• Terminal Output: invoice_2026.pdf.sh: ASCII text

• Observed Result: The file inspection utility confirms that the underlying payload is plain ASCII text/script code rather than a legitimate Adobe PDF document structure. When operating systems hide known extensions, users visually see invoice_2026.pdf and unwittingly trigger script execution.